

Classificação de Tumores Cerebrais em Imagens de Ressonância Magnética (RM) com Inteligência Artificial

Instituição: Instituto Federal de Minas Gerais (IFMG) - Campus Ouro Branco

Curso: Bacharelado em Sistemas de Informação

Disciplina: Inteligência Artificial - 2026

Aluno: Caíque Augusto

Professor: Ângelo Magno

1. Introdução e Objetivo

O diagnóstico precoce e preciso de tumores cerebrais é de extrema importância para o sucesso do tratamento clínico e para a sobrevivência dos pacientes. A análise manual de imagens de Ressonância Magnética (RM) por radiologistas é uma tarefa complexa, demorada e sujeita à variabilidade interobservador. Nesse contexto, sistemas de Inteligência Artificial baseados em Aprendizado de Máquina (Machine Learning) e Aprendizado Profundo (Deep Learning) surgem como ferramentas de suporte ao diagnóstico médico, capazes de identificar padrões sutis nas imagens de forma rápida e reproduzível.

O objetivo principal deste trabalho é implementar, avaliar e comparar diferentes abordagens de Aprendizado de Máquina para classificar imagens de RM cerebral em quatro categorias:

1. **Glioma:** Tumor cerebral primário originado nas células gliais.
2. **Meningioma:** Tumor que se desenvolve nas meninges (membranas que envolvem o cérebro).
3. **Tumor Hipofisário (Pituitary):** Tumor que afeta a glândula hipófise.
4. **Sem Tumor (No Tumor):** Imagens de cérebros saudáveis, sem presença de neoplasias.

Para isso, foram realizados **8 experimentos sistemáticos** comparando duas abordagens distintas de modelagem:

- **Redes Neurais Convolucionais (CNNs)** desenvolvidas com a API Keras/TensorFlow.
- **Support Vector Machines (SVMs)** combinados com extratores de características de textura baseados em **HOG (Histogram of Oriented Gradients)** utilizando a biblioteca Scikit-Learn.

Os experimentos envolveram a avaliação das métricas de **Acurácia**, **Precisão**, **Recall** (Sensibilidade) e **F1-Score (F-Measure)**, além da análise de matrizes de confusão e custos computacionais (tempo de treinamento).

2. Descrição do Dataset e Pré-processamento

2.1 O Dataset Brain Tumor MRI

O conjunto de dados utilizado foi o **Brain Tumor MRI Dataset** obtido da plataforma Kaggle. Ele contém um total de **7.200 imagens** de RM cerebral organizadas de maneira equilibrada nas quatro classes mencionadas. A distribuição entre os conjuntos foi realizada de forma estratificada:

- **Treinamento (Training):** 4.480 imagens (1.120 por classe)
- **Validação (Validation):** 1.120 imagens (280 por classe)
- **Teste (Testing):** 1.600 imagens (400 por classe)

A estrutura balanceada do conjunto de teste (exatamente 400 imagens por classe) permite uma avaliação direta e robusta das métricas globais (médias macro e weighted).

2.2 Pré-processamento para as CNNs

Para a alimentação das Redes Neurais Convolucionais, o fluxo de pré-processamento envolveu:

1. **Redimensionamento:** Redução das dimensões originais das imagens para 128×128 pixels.
2. **Normalização (Rescaling):** Divisão dos valores dos pixels (originalmente de 0 a 255) por 255.0 para trazê-los para o intervalo $[0, 1]$, facilitando a convergência do otimizador.
3. **Pipeline TF Data:** Criação de objetos `tf.data.Dataset` otimizados com `prefetch` e carregamento assíncrono (`AUTOTUNE`).
4. **Data Augmentation (Apenas no Experimento 4):** Aplicação de transformações geométricas (espelhamento horizontal, rotação suave, zoom e translação) nas imagens de treino de forma dinâmica para reduzir o overfitting.

2.3 Extração de Características HOG para os SVMs

Os classificadores SVM não recebem tensores de imagem diretamente. Para eles, extraímos características de textura e forma usando **HOG (Histogram of Oriented Gradients)**:

1. **Conversão para Escala de Cinza:** Redução dos canais de cor de RGB para monocromático.
2. **Redimensionamento:** Ajuste para 64×64 pixels.
3. **Configuração do HOG:**
 - Orientações do gradiente (`orientations`): 9.
 - Tamanho da célula (`pixels_per_cell`): 8×8 pixels.
 - Células por bloco (`cells_per_block`): 2×2 blocos.
 - Normalização de bloco: L2-Hys.
4. Isso converte cada imagem em um vetor unidimensional de **1.764 características estruturais**, que posteriormente são padronizadas usando o `StandardScaler` do Scikit-Learn antes de serem fornecidas ao SVM.

3. Metodologia e Experimentos

Foram configurados e executados 8 experimentos variando técnicas, arquiteturas, regularização e hiperparâmetros, conforme detalhado a seguir:

3.1 Experimentos com Redes Neurais Convolucionais (CNN)

- **Experimento 1 (CNN Básica - Modelo de Referência):**
 - *Arquitetura:* Três blocos convolucionais convolutivos sequenciais com 32, 64 e 128 filtros (tamanho de kernel 3×3 com ativação ReLU), seguidos por camadas de `MaxPooling2D`. Após a convolução, é aplicada uma camada de `GlobalAveragePooling2D`, seguida de uma camada densa oculta de 128 neurônios (ReLU) e a camada de saída Softmax com 4 neurônios. Sem qualquer tipo de regularização.

- *Treinamento*: 15 épocas, otimizador Adam ($lr=0.001$), perda de entropia cruzada categórica esparsa.
- **Experimento 2 (CNN com Dropout 20%)**:
 - *Arquitetura*: Idêntica à CNN Básica, porém com a inclusão de uma camada de **Dropout** com taxa de 20% (0.20) posicionada logo após a camada densa oculta de 128 neurônios. Objetivo de prevenir o co-adaptamento dos neurônios densos.
- **Experimento 3 (CNN com Batch Normalization e Dropout 50%)**:
 - *Arquitetura*: Adiciona camadas de **BatchNormalization** logo após cada uma das convoluções e após a camada densa de 128 neurônios. O dropout após a camada densa é elevado para 50% (0.50). Remove o viés (**use_bias=False**) nas camadas que precedem o Batch Normalization.
- **Experimento 4 (CNN com Data Augmentation, Regularização L2 e Early Stopping)**:
 - *Arquitetura*: Baseada na CNN básica, mas com a adição de um bloco inicial de *Data Augmentation* (camadas Keras que realizam rotações, translações, inversões e zoom dinâmicos em tempo de treino). Além disso, adiciona regularização L2 com fator 10^{-4} (**kernel_regularizer=l2(1e-4)**) em todas as camadas convolucionais e na camada densa.
 - *Callbacks*: Early Stopping com paciência de 4 épocas monitorando a perda de validação (**val_loss**) e decaimento da taxa de aprendizado (**ReduceLROnPlateau**) com paciência de 2 épocas. Limite máximo de 25 épocas.

3.2 Experimentos com Support Vector Machines (SVM)

- **Experimento 5 (SVM Linear - $C=1$)**:
 - *Metodologia*: Extração das características HOG (64×64), padronização com **StandardScaler** e treinamento de um SVM com **kernel linear** e parâmetro de regularização $C=1$.
 - **Experimento 6 (SVM RBF - $C=1$)**:
 - *Metodologia*: Utiliza o **kernel RBF (Radial Basis Function)**, que mapeia as características HOG para um espaço dimensional infinito não-linear. Parâmetro $C=1$ e $\gamma = "scale"$.
 - **Experimento 7 (SVM RBF - $C=10$)**:
 - *Metodologia*: Utiliza o kernel RBF com maior tolerância a erros e maior penalização no treinamento através de $C=10$. Isso incentiva o classificador a ajustar melhor os limites de decisão.
 - **Experimento 8 (SVM RBF com PCA e Balanceamento)**:
 - *Metodologia*: Para reduzir a dimensionalidade dos vetores HOG (e acelerar o processo), aplica-se a redução por **PCA (Análise de Componentes Principais)** para extrair as 256 principais componentes que explicam a maior variabilidade. Utiliza-se kernel RBF, $C=10$ e pesos de classe balanceados (**class_weight="balanced"**) para dar a mesma importância teórica a todas as categorias.
-

4. Resultados Experimentais

Os resultados obtidos em cada um dos 8 experimentos, coletados no conjunto de teste independente (1.600 imagens) e no conjunto de validação, são resumidos na tabela a seguir:

Exp.	Técnica	Modelo / Configuração	Val. Acc	Test Acc	Prec. Macro	Rec. Macro	F1-Score Macro	Falsos Negativos*	Tempo Treino	Épocas
1	CNN	CNN básica (Sem regularização)	0.8330	0.7194	0.7221	0.7194	0.7009	214	128.8s	15
2	CNN	CNN com Dropout 20%	0.8188	0.7094	0.7319	0.7094	0.7064	88	119.0s	15
3	CNN	CNN Batch Norm e Dropout 50%	0.8384	0.5256	0.7955	0.5256	0.4676	26	112.4s	15
4	CNN	CNN Augmentation + L2 + EarlyStop	0.3643	0.3244	0.3615	0.3244	0.2379	883	45.6s	5
5	SVM	SVM Linear C=1 (HOG)	0.8589	0.8244	0.8229	0.8244	0.8213	67	6.4s	-
6	SVM	SVM RBF C=1 (HOG)	0.9054	0.8531	0.8547	0.8531	0.8479	72	10.8s	-
7	SVM	SVM	0.936	0.894	0.896	0.894	0.891	54	12.5s	-

		RBF C=10 (HOG)	6	4	4	4	0			
8	SVM	SVM RBF C=10 + PCA 256 + Balan.	0.9411	0.8925	0.8941	0.8925	0.8895	65	3.5s	-

* Falsos Negativos indicam o número total de imagens de tumores (glioma, meningioma ou tumor hipofisário) classificadas erroneamente como "notumor". Esta métrica é crítica no ambiente de saúde, pois um falso negativo pode atrasar um tratamento médico vital.

Análise das Métricas por Classe (Melhor Abordagem: Experimento 7 - SVM RBF C=10)

No Experimento 7, o relatório de classificação por classe no conjunto de teste revelou excelente equilíbrio:

- **Glioma:** Precisão = 92.6%, Recall = 72.0%, F1-Score = 81.0% (suporte = 400)
- **Meningioma:** Precisão = 85.4%, Recall = 87.8%, F1-Score = 86.6% (suporte = 400)
- **Sem Tumor (No Tumor):** Precisão = 88.1%, Recall = 100.0%, F1-Score = 93.7% (suporte = 400)
- **Tumor Hipofisário:** Precisão = 92.5%, Recall = 98.0%, F1-Score = 95.1% (suporte = 400)

5. Análise Crítica e Conjecturas

5.1 Superioridade dos Modelos SVM baseados em HOG

Os experimentos mostraram de forma clara que **os modelos SVM baseados em HOG superaram as Redes Neurais Convolucionais (CNNs)** em quase todas as métricas no conjunto de teste. O Experimento 7 atingiu o maior F1-Score Macro (89.10%) e a maior acurácia (89.44%).

Podem-se fazer as seguintes conjecturas sobre essa diferença de desempenho:

1. Limitação de Épocas e Recursos no Treinamento de CNNs:

As CNNs foram configuradas para treinar por apenas 15 épocas. Redes neurais convolucionais construídas do zero possuem milhões de parâmetros livres e necessitam de muito mais tempo de otimização (normalmente de 50 a 150 épocas) para aprender representações hierárquicas de baixo, médio e alto nível (bordas, texturas, formas geométricas complexas) capazes de generalizar bem. O SVM, por outro lado, trabalha com características estruturadas HOG pré-calculadas. Ele não precisa "aprender" a extrair características das imagens; o HOG já entrega os descritores de bordas e orientações bem definidos. O SVM precisa apenas encontrar o hiperplano de separação linear ou não-linear ideal, o que é um problema de otimização convexo muito mais rápido e estável.

2. Vazamento de Dados (Data Leakage) na Validação:

Nos experimentos de CNN, observamos uma lacuna considerável entre a acurácia de validação e a de teste. Por exemplo, a CNN básica atingiu 83.30% de validação, mas caiu para 71.94% no teste. Isso ocorre muito frequentemente em datasets médicos onde imagens sequenciais do mesmo paciente estão presentes no conjunto de dados. Se a divisão de treino e validação for feita de forma puramente aleatória (como é feito no `train_test_split`), fatias diferentes da RM do mesmo cérebro de um mesmo paciente terminam em ambos os conjuntos. A rede neural decora características individuais do paciente (como o formato do crânio) em vez de aprender os sinais do tumor em si. Quando o modelo é avaliado no conjunto de teste oficial (`Testing/`), que contém pacientes totalmente novos e independentes, o desempenho cai.

3. Regularização Excessiva e Subajuste (Underfitting) na CNN (Experimento 4):

No Experimento 4, a combinação de Data Augmentation, L2 regularização e paciência reduzida para o Early Stopping fez com que o modelo sofresse de subajuste grave. O treinamento foi interrompido na época 5 com apenas 32.44% de acurácia. O aumento de dados tornou o treinamento difícil demais para a capacidade de representação do modelo simples em poucas épocas, gerando um modelo que convergiu para uma heurística ingênua (classificar quase tudo como "notumor", gerando 883 falsos negativos).

4. Habilidade Descritiva do HOG para RMs Cerebrais:

Ressonâncias magnéticas de crânio possuem estruturas anatômicas geométricas muito rígidas (a forma arredondada do crânio, a simetria dos hemisférios cerebrais). Alterações de textura e quebras nessa simetria são fortes indicadores de anomalias (tumores). O algoritmo HOG, ao capturar distribuições de orientações de gradientes em blocos locais, é excepcionalmente bom para detectar essas perturbações de contorno e textura geométrica. Com isso, os dados passados para o SVM já possuíam um alto teor de discriminabilidade biológica.

5.2 O Caso do Experimento 3 e Falsos Negativos

Embora a CNN do Experimento 3 tenha apresentado uma acurácia geral baixa (52.56%), ela chamou atenção por ter apenas **26 falsos negativos** (tumores classificados como "notumor").

Contudo, uma análise mais atenta revela que isso **não representa um bom modelo**:

- O recall para a classe `meningioma` foi de 97.5% (identificou quase todos), mas a precisão foi de apenas 35.1%.
- O recall para a classe `pituitary` foi de apenas 2.75%.
- A rede sofreu um colapso de aprendizado, classificando a grande maioria das imagens como pertencentes a uma única classe de tumor (meningioma). Por classificar quase todas as imagens de teste como "algum tipo de tumor", o modelo naturalmente obteve um baixo número de falsos negativos estruturais (já que quase nada era predito como "notumor"). Porém, o modelo é inutilizável clinicamente devido ao altíssimo índice de falsos alarmes e à incapacidade de diferenciar os tipos de tumor.

5.3 Custo-Benefício da Redução de Dimensionalidade (PCA)

O **Experimento 8 (SVM RBF PCA Balanceado)** demonstrou o poder de engenharia clássica de machine learning:

- Ao aplicar o PCA, reduzimos o número de características estruturais HOG de 1.764 para 256 componentes principais.
- Isso diminuiu o tempo de treinamento de **12.49 segundos** (Experimento 7) para apenas **3.47 segundos** (uma redução de **72%** no custo computacional).
- A perda de desempenho foi insignificante: a acurácia passou de 89.44% para 89.25%, e o F1-Score macro variou de 89.10% para 88.95%.
- Esta abordagem é altamente recomendada para produção, pois economiza memória e poder de processamento mantendo o mesmo nível de precisão de diagnóstico.

6. Códigos Utilizados como Base

Abaixo estão os trechos de código estruturais que serviram de base para a execução dos principais experimentos deste projeto.

6.1 Pipeline de Extração HOG (Base para SVM)

```
from PIL import Image
import numpy as np
from skimage.feature import hog

def extrair_hog_imagem(caminho: str) -> np.ndarray:
    with Image.open(caminho) as imagem:
        imagem = imagem.convert("L") # Escala de cinza
        imagem = imagem.resize(
            (64, 64), # Resolução reduzida
            Image.Resampling.BILINEAR,
        )
        matriz = np.asarray(imagem, dtype=np.float32) / 255.0

    caracteristicas = hog(
        matriz,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        block_norm="L2-Hys",
        feature_vector=True,
    )
    return caracteristicas.astype(np.float32)
```

6.2 Construção e Treinamento do SVM RBF C=10 (Experimento 7)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

# Definição do Pipeline de Classificação
```



```

pipeline_exp_7 = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(
        kernel="rbf",
        C=10,
        gamma="scale",
        cache_size=1500
    ))
])

# Treinamento com as características HOG
pipeline_exp_7.fit(X_train_hog, y_train_hog)

# Predição e Avaliação
y_test_predito = pipeline_exp_7.predict(X_test_hog)

```

6.3 Modelo Base da CNN no Keras (Experimento 1)

```

import tensorflow as tf

def construir_cnn_experimento_1():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(128, 128, 3)),
        tf.keras.layers.Rescaling(1.0 / 255),

        tf.keras.layers.Conv2D(32, kernel_size=3, padding="same", activation="relu"),
        tf.keras.layers.MaxPooling2D(),

        tf.keras.layers.Conv2D(64, kernel_size=3, padding="same", activation="relu"),
        tf.keras.layers.MaxPooling2D(),

        tf.keras.layers.Conv2D(128, kernel_size=3, padding="same", activation="relu"),
        tf.keras.layers.MaxPooling2D(),

        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dense(4, activation="softmax", dtype="float32")
    ])

# Compilação e Treinamento
modelo = construir_cnn_experimento_1()
modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
)
modelo.fit(train_ds, validation_data=val_ds, epochs=15)

```

7. Conclusão

Este trabalho realizou um estudo comparativo abrangente para a classificação de quatro tipos de imagens de RM cerebral usando Redes Neurais Convolucionais e algoritmos SVM com extratores estruturais HOG.

O modelo **SVM com kernel RBF e C=10 (Experimento 7)** foi a melhor abordagem para o problema, alcançando **89,44% de acurácia** e **89,10% de F1-Score macro** no conjunto de teste independente. O **Experimento 8 (com redução PCA para 256)** mostrou-se a alternativa mais eficiente em termos computacionais, reduzindo em 72% o tempo de treinamento sem perdas significativas de performance.

As CNNs sofreram com limitações de convergência devido ao baixo número de épocas de treinamento e mostraram vulnerabilidade ao vazamento de dados na validação cruzada aleatória, apontando para a importância de estratégias mais avançadas de divisão de dados no ambiente médico (como agrupamento por paciente - *GroupKFold*).

8. Vídeo de Apresentação

O vídeo explicativo detalhando os códigos, a metodologia e os resultados pode ser acessado através do link a seguir:

https://youtu.be/G8nzDF_gPbQ